

Procure-to-Pay (P2P) ERP System

Full-Stack Web Application

Capstone Project Documentation

KIIT Capstone Project | April 2026

Tech Stack: Python (Flask) • HTML5 • CSS3 • JavaScript

Name:	<u>Atharv Sheersh Chaturvedi</u>
Roll Number:	<u>23053034</u>
Batch / Program:	<u>BTech/CSE</u>

1. Problem Statement

Small and mid-sized enterprises often manage procurement operations through disconnected spreadsheets, paper-based purchase orders, and manual invoice tracking. This fragmented approach leads to delays in vendor onboarding, difficulty tracking order statuses, missed payment due dates, and poor visibility into inventory levels. The absence of a centralised digital system results in financial leakage and operational inefficiency.

This project addresses these pain points by delivering a unified web-based ERP module covering the complete Procure-to-Pay (P2P) lifecycle — from vendor registration and purchase order creation through invoice processing and inventory management.

2. Solution & Features

Dashboard	Real-time KPI cards showing active vendors, total PO value, unpaid invoices, and low-stock alerts alongside recent purchase order activity.
Vendor Management	Register and manage vendor master data including company name, category, contact details, and active/inactive status. Supports deletion of obsolete records.
Purchase Orders	Create purchase orders linked to active vendors with automatic total calculation. Track order lifecycle: Pending → In Transit → Received → Cancelled.
Invoice Management	View all invoices with due dates and payment status. One-click 'Mark as Paid' action updates invoice status in real time.
Inventory / Stock Register	Maintain a warehouse stock register with item categories, quantities, and reorder levels. Automatic low-stock alert badges highlight items requiring replenishment.

3. Application Screenshots

The application consists of five main screens accessible via the sidebar navigation:

Dashboard	Displays four KPI cards (active vendors, total POs, total PO value in INR, unpaid invoice amount) and a table of recent purchase orders with status badges. Quick-action buttons link to each module.
Vendors	A searchable table lists all vendors with ID, name, category, contact, email, and status. An 'Add Vendor' modal form allows instant registration. A trash icon removes inactive vendors.
Purchase Orders	All POs displayed in a full-width table. A 'Create PO' modal calculates order totals dynamically as the user types quantity and unit price. A dropdown updates PO status inline.
Invoices	Invoice list with PO reference, vendor, amount (formatted in INR), due date, and payment status badge. Unpaid invoices show a 'Mark Paid' action button.
Inventory	Stock register with item ID, category, quantity, unit, reorder level, and warehouse. Items at or below reorder level display a red 'Low Stock' badge automatically.

4. Tech Stack

Layer	Technology	Purpose
Backend	Python 3 + Flask	REST API server, route handling, JSON data persistence
Frontend	HTML5 + CSS3	Semantic markup, custom responsive design with CSS
Frontend	Vanilla JavaScript	Fetch API for async REST calls, DOM manipulation, n
Icons	Font Awesome 6	UI iconography via CDN
Storage	JSON file (data.json)	Lightweight in-memory-style data store; no database r
Fonts	Segoe UI / system-ui	Native system font stack for fast rendering

5. Unique Points

■ Zero-dependency frontend

Built entirely with vanilla JS and CSS — no React, Vue, or jQuery. This keeps the bundle size at zero and demonstrates mastery of web fundamentals.

■ REST API architecture

Flask exposes clean RESTful endpoints (GET, POST, PATCH, DELETE) that decouple the frontend from the backend, making the app easy to extend or re-skin.

■ Dynamic total calculation

Purchase order totals are computed client-side in real time as the user types, improving the data-entry experience without a server round-trip.

■ Automatic low-stock alerts

Inventory items are flagged with a red badge automatically when stock falls to or below the configured reorder level — no manual review required.

■ File-based persistence

Uses a JSON file as the data store, making the app fully portable and runnable with a single command — no database installation needed.

■ Responsive sidebar layout

The sidebar collapses to icon-only mode on small screens using pure CSS media queries, making the app usable on tablets and mobile devices.

6. Future Improvements

- Migrate from JSON file storage to SQLite or PostgreSQL for concurrent access and proper querying.
- Add user authentication with role-based access (Admin, Procurement Officer, Finance) using Flask-Login.
- Implement email notifications for purchase order approvals and invoice due-date reminders.
- Add PDF export of individual purchase orders and invoices directly from the browser.
- Integrate Chart.js visualisations on the dashboard for spend analytics and vendor performance trends.
- Build a full audit trail / activity log to track who created or modified each record.
- Add a search and filter bar across all tables for large datasets.
- Deploy on a cloud platform (Render, Railway, or AWS EC2) with a public URL.

7. API Endpoint Reference

Method	Endpoint	Description
GET	/api/dashboard	KPI stats for dashboard cards and recent POs
GET	/api/vendors	Retrieve all vendor records
POST	/api/vendors	Create a new vendor
DELETE	/api/vendors/<id>;	Delete a vendor by ID
GET	/api/purchase-orders	Retrieve all purchase orders
POST	/api/purchase-orders	Create a new purchase order
PATCH	/api/purchase-orders/<id>/status	Update PO status
GET	/api/invoices	Retrieve all invoices
PATCH	/api/invoices/<id>/pay	Mark an invoice as paid
GET	/api/inventory	Retrieve all inventory items
POST	/api/inventory	Add a new inventory item

8. Project Structure

```
capstone_project/ app.py ← Flask application & REST API routes requirements.txt ←  
Python dependencies (Flask only) data.json ← Auto-generated JSON data store README.md  
← Project documentation templates/ base.html ← Base layout with sidebar navigation  
index.html ← Dashboard page vendors.html ← Vendor management page purchase_orders.html  
← Purchase orders page invoices.html ← Invoice management page inventory.html ←  
Inventory / stock register page static/ css/style.css ← Complete custom stylesheet  
js/main.js ← Modal helper functions
```

9. How to Run

Step 1 — Unzip	Extract the submitted ZIP file to a local folder.
Step 2 — Install Python	Ensure Python 3.8+ is installed. Verify with: <code>python --version</code>
Step 3 — Install Flask	Open a terminal in the project folder and run: <code>pip install -r requirements.txt</code>
Step 4 — Start server	Run the Flask development server: <code>python app.py</code>
Step 5 — Open browser	Visit <code>http://127.0.0.1:5000</code> to use the application.

The application uses a JSON file (`data.json`) as its data store. This file is created automatically on first run with sample data. All CRUD operations persist between server restarts. No database setup is required — the project runs with a single **pip install** and **python app.py**.